UNIT 4 · TOPIC 4.2

Authentication

Assignment, Answer Key

Teacher answer key, 39 points total.

Practice Questions

4.2.a · Passwords and password policies

1. According to NIST SP 800-63B, which practice is recommended when a user creates a new password? (1 pt)

- ○ A. Require at least one uppercase letter, one digit, and one symbol
- ○ B. Force the user to change the password every 90 days
- **● C. Check the candidate against a list of known-compromised passwords and reject matches**
- ○ D. Block copy-paste to prevent shoulder surfing

Why: NIST 800-63B dropped mandatory complexity rules and periodic rotation, but does require checking new passwords against known-breach lists.

2. Which of the following are valid reasons an organization enforces an account lockout policy? (Select ALL that apply.) (2 pts)

- **■ A. Prevents automated online guessing attacks**
- □ B. Eliminates the need for password hashing
- **■ C. Forces attackers to slow down their credential testing**
- **■ D. Makes brute-force infeasible in an interactive login scenario**

Why: Lockout blocks fast online guessing and slows credential testing. It does not replace hashing, which protects the stored credential database offline.

3. An attacker obtains a copy of a site's password database. The passwords are stored with bcrypt. Why does this algorithm make cracking harder than MD5? (1 pt)

- ○ A. Bcrypt produces longer hashes
- ○ B. Bcrypt hashes are reversible, allowing easier comparison
- ○ C. Bcrypt requires a network connection to verify
- **● D. Bcrypt is intentionally slow, making each guess computationally expensive**

Why: Bcrypt is a deliberately slow password-hashing algorithm; each guess takes orders of magnitude longer than with MD5, making large-scale offline cracking impractical.

4. What is the primary purpose of a cryptographic salt in password storage? (1 pt)

- ○ A. To make the password longer before the user sees it
- ○ B. To encrypt the password so it can be decrypted later
- **● C. To ensure two users with the same password have different stored hashes**
- ○ D. To enforce complexity rules automatically

Why: A salt is a unique random value per user. Two users with identical passwords produce different hashes, defeating precomputed rainbow-table attacks.

5. Name the two attack types that increasing password LENGTH most directly defends against. (1 pt)

Acceptable answers: brute-force; brute force; dictionary; dictionary attack; brute-force and dictionary

Why: Length exponentially expands the search space for brute-force attacks and reduces overlap with dictionary word lists.

4.2.b · Multi-factor authentication

6. A user logs in with a password, then enters a memorized answer to a security question. Is this true multi-factor authentication? (1 pt)

- **● A. No, both credentials are 'something you know', the same factor category**
- ○ B. Yes, two credentials were entered
- ○ C. Yes, because a security question is a possession factor
- ○ D. No, security questions are never allowed in authentication

Why: A password and a security-question answer are both knowledge factors. True MFA requires two DIFFERENT factor categories, not just two credentials.

7. Which factor category does a hardware security key (like a FIDO2/YubiKey device) belong to? (1 pt)

- ○ A. Something you know
- **● B. Something you have (possession)**
- ○ C. Something you are (inherence)
- ○ D. It is not a factor at all

Why: A hardware key is a physical object the user possesses, the possession factor category.

8. Select ALL statements that are TRUE about SMS-based one-time codes used for MFA. (2 pts)

- **■ A. They are vulnerable to SIM-swapping attacks**
- □ B. They are immune to phishing because the code changes every time
- **■ C. They can be intercepted or relayed by an adversary-in-the-middle phishing kit**
- **■ D. They still combine a knowledge factor (password) with a possession factor (phone)**

Why: SMS codes are possession-factor evidence but are vulnerable to SIM swapping and real-time phishing relay; a changing code does not prevent an attacker from relaying it live to the real site.

9. An attacker who already has a victim's password sends them ten push-approval MFA prompts in a row until the victim taps Approve just to stop the notifications. What is this attack called? (1 pt)

- ○ A. SIM swapping
- **● B. MFA fatigue (push bombing)**
- ○ C. TOTP replay
- ○ D. Credential stuffing

Why: Repeatedly sending push prompts until an annoyed or confused user approves one is MFA fatigue, also called push bombing.

10. Why is FIDO2/WebAuthn considered phishing-resistant compared to a typed authenticator-app code? (1 pt)

- **● A. It cryptographically binds the response to the legitimate site's exact domain, so a fake login page cannot obtain a valid response**
- ○ B. It never requires a second device
- ○ C. It does not require a password at all, ever
- ○ D. It uses a longer numeric code than TOTP

Why: FIDO2/WebAuthn performs a domain-bound cryptographic challenge-response, so even a convincing fake site cannot capture a usable credential, unlike a typed code, which a user can be tricked into entering anywhere.

11. Name the attack in which an adversary tricks or bribes a mobile carrier into porting a victim's phone number to an attacker-controlled SIM card, in order to intercept SMS-based MFA codes. (1 pt)

Acceptable answers: sim swap; sim swapping; sim-swap; sim swap attack; simjacking

Why: SIM swapping transfers the victim's phone number to a SIM the attacker controls, letting the attacker receive SMS MFA codes intended for the victim.

4.2.c · Session security

12. An attacker sends a victim a link containing a session ID the attacker already knows. The victim logs in and the server keeps using that same ID. Which attack is this? (1 pt)

- ○ A. Session hijacking
- ○ B. SQL injection
- **● C. Session fixation**
- ○ D. Brute force

Why: The attacker supplied the session ID BEFORE the victim authenticated, this is session fixation, distinct from hijacking a token that was already live.

13. Which cookie attribute prevents JavaScript (including injected XSS scripts) from reading the session cookie? (1 pt)

- **● A. HttpOnly**
- ○ B. Secure
- ○ C. SameSite
- ○ D. Max-Age

Why: The HttpOnly flag hides the cookie from JavaScript entirely, so even a successful XSS injection cannot read and exfiltrate it.

14. Select ALL statements that are TRUE about session timeouts. (2 pts)

- **■ A. An idle timeout ends a session after a period of no activity**
- **■ B. An absolute timeout ends a session after a fixed total duration regardless of activity**
- □ C. A session with no timeout configured poses no additional risk
- **■ D. Timeouts limit how long a stolen or fixated token remains useful**

Why: Idle and absolute timeouts are distinct expiration mechanisms, and both limit the damage a leaked token can do. A session with no timeout at all is a real risk, not a neutral choice.

15. A user clicks 'Log Out' and their browser cookie is deleted, but the server never marks the session record as invalid. What is the consequence? (1 pt)

- ○ A. No consequence; deleting the cookie is sufficient
- ○ B. The user's password is exposed
- **● C. A token captured earlier by an attacker remains valid and usable**
- ○ D. The session is automatically regenerated

Why: Logout must invalidate the session server-side. If it does not, any copy of the token an attacker already captured continues to work, regardless of the client-side cookie deletion.

16. Name the defensive technique of issuing a brand-new session ID at login (and at privilege change) that specifically defeats session fixation. (1 pt)

Acceptable answers: session regeneration; regenerate session id; session id regeneration; regenerating the session id

Why: Session regeneration discards any attacker-planted ID at the moment of login, so fixation fails even if the attacker set an ID on the victim's browser beforehand.

17. A session cookie is sent only over HTTPS due to one cookie flag, but is still readable by page JavaScript because a different flag is missing. Which flag is MISSING? (1 pt)

- ● **A. HttpOnly**
- ○ B. Secure
- ○ C. Domain
- ○ D. Path

Why: Secure restricts transmission to HTTPS; HttpOnly is the separate flag that hides the cookie from JavaScript. HTTPS alone does not stop an XSS-based cookie read.

4.2.d · Least privilege and access control models

18. What does the principle of least privilege (PoLP) require? (1 pt)

- ● **A. A subject should be granted only the minimum access needed for its job function**
- ○ B. Every account should have administrator access by default
- ○ C. All employees in the same department should share one account
- ○ D. Access should never be reviewed once granted

Why: PoLP means granting only the minimum access required to perform the current job function, nothing more.

19. A file's owner directly decides who else may read or write it, without any central authority involved. Which access control model is this? (1 pt)

- ○ A. MAC
- ○ B. RBAC
- ○ C. ABAC
- ● **D. DAC**

Why: Discretionary Access Control (DAC) lets the resource owner grant or revoke access directly; classic Unix file permissions are the standard example.

20. A system grants access dynamically based on the user's department, the sensitivity label of the resource, and whether the request happens during business hours. Which model is this? (1 pt)

- ● **A. ABAC**
- ○ B. MAC
- ○ C. RBAC
- ○ D. DAC

Why: Attribute-Based Access Control (ABAC) evaluates attributes of the user, resource, and environment at request time to make a dynamic access decision.

21. Select ALL statements that are TRUE about over-privileged accounts. (2 pts)

- ■ **A. They increase the blast radius if the account is compromised**
- □ B. They eliminate insider-threat risk because trusted employees hold them
- ■ **C. Malware executed under an over-privileged session inherits that account's broad access**
- ■ **D. They make ransomware propagation to additional systems more likely**

Why: Over-provisioning expands blast radius, lets malware inherit broader privileges, and increases ransomware spread risk. Trust does not eliminate insider-threat or compromise risk.

22. Name the term for the silent accumulation of unnecessary access rights over time as an employee's role changes. (1 pt)

Acceptable answers: privilege creep; access creep; permission creep

Why: Privilege creep is the accumulation of unneeded access over time; it is mitigated by periodic access reviews/recertification.

23. Which statement correctly distinguishes RBAC from ABAC? (1 pt)

○ A. RBAC and ABAC are the same thing, just different names

● **B. RBAC assigns static permissions via roles; ABAC evaluates dynamic attributes at request time**

○ C. RBAC is only used in government systems; ABAC is only used in enterprises

○ D. ABAC does not support access denial, only approval

Why: RBAC attaches fixed permissions to a role and users inherit that role's access. ABAC makes a dynamic, per-request decision based on attributes of user, resource, and environment.

Coding Assignments (Python 3)

4.2.a · Passwords and password policies

3 pts · auto-graded on the site

Coding problem, 4.2.a: Passwords and password policies

Skill: Mitigate Risk · **Maps to:** 4.2.A · **Language:** Python 3 (pytest)

Problem

A system administrator has defined a password policy (minimum length, complexity requirements, a banned/breached-password list, and a history of previously used passwords). Build the checker that evaluates a candidate password against that policy and reports every rule it violates, so the login form can show the user exactly what to fix.

Function contract

Implement `validate_password(password: str, policy: dict) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `password`, a string, the candidate password.
- `policy`, a dict. Example: ``python policy = { "min_length": 12, "require_upper": True, "require_lower": True, "require_digit": True, "require_symbol": True, "banned_list": ["password123", "letmein", "qwerty123456"], "history": ["OldPass2023!", "OldPass2024!"], } Any key may be omitted; a missing key means that rule is not enforced (see interface.md` for exact defaults).

Output

A `list[str]` of violated rule names, in the fixed order defined in `interface.md`: `too_short`, `no_upper`, `no_lower`, `no_digit`, `no_symbol`, `in_banned_list`, `reused_password`. An empty list `[]` means the password is fully compliant.

Examples

```
validate_password("abc", {"min_length": 12, "require_upper": True, "require_digit": True})
    -> ["too_short", "no_upper", "no_digit"]
```

```
validate_password("Password123", {"banned_list": ["password123"]})
    -> ["in_banned_list"]

validate_password(
    "Tr0ub4dor&3xtra!",
    {"min_length": 12, "require_upper": True, "require_lower": True,
     "require_digit": True, "require_symbol": True,
     "banned_list": ["password123"], "history": ["OldPass2023!"]},
) -> []
```

Constraints

- Deterministic, offline, standard library only.
- Missing policy keys are treated as "not required" (see `interface.md` for exact defaults).
- Violated rule names must be returned in the fixed order given above, even when several rules are violated at once.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def validate_password(password: str, policy: dict) -> list:
    min_length = policy.get("min_length", 0)
    require_upper = policy.get("require_upper", False)
    require_lower = policy.get("require_lower", False)
    require_digit = policy.get("require_digit", False)
    require_symbol = policy.get("require_symbol", False)
    banned_list = policy.get("banned_list", [])
    history = policy.get("history", [])

    violations = []

    if len(password) < min_length:
        violations.append("too_short")

    if require_upper and not any(c.isupper() for c in password):
        violations.append("no_upper")

    if require_lower and not any(c.islower() for c in password):
        violations.append("no_lower")

    if require_digit and not any(c.isdigit() for c in password):
        violations.append("no_digit")

    if require_symbol and not any(not c.isalnum() for c in password):
        violations.append("no_symbol")

    if any(password.lower() == banned.lower() for banned in banned_list):
        violations.append("in_banned_list")

    if any(password == prior for prior in history):
        violations.append("reused_password")

    return violations
```

4.2.b · Multi-factor authentication

3 pts · auto-graded on the site

Coding problem, 4.2.b: Classify a login attempt's MFA strength

Skill: Mitigate Risk · **Maps to:** 4.2.A, 4.2.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives, for each login attempt, the list of authentication factors that were presented (e.g. `["password", "sms_code"]`). Build the classifier that determines whether the attempt satisfies true multi-factor authentication, and if so, whether it relies on a phishing-resistant possession factor or a phishable one.

Categories of result:

- `"single_factor"`, the presented factors, once mapped to their factor category (knowledge / possession / inherence), cover fewer than two distinct categories. This includes presenting only one factor, presenting several factors from the *same* category (e.g. two passwords), an empty list, or a list of only unrecognized factor strings.
- `"mfa_phishing_resistant"`, two or more distinct factor categories are present, and at least one of the presented factors is a phishing-resistant possession factor (`hardware_key` or `fido2`).
- `"mfa_phishable"`, two or more distinct factor categories are present, but no phishing-resistant possession factor was used (e.g. it relies on `sms_code`, `totp_code`, `authenticator_app`, `push_approved`, or `backup_code` for the possession factor).

The exact factor-to-category mapping and full rule set are defined in `interface.md`, implementations must follow it precisely.

Function contract

Implement `solve(factors: list[str]) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of strings, each a recognized or unrecognized factor name. May be empty.

Output

One of: `"single_factor"`, `"mfa_phishable"`, `"mfa_phishing_resistant"`.

Examples

```
["password"]                          -> "single_factor"
["password", "password"]              -> "single_factor"
["password", "security_question"]     -> "single_factor"
["password", "sms_code"]              -> "mfa_phishable"
["password", "push_approved"]         -> "mfa_phishable"
["password", "hardware_key"]          -> "mfa_phishing_resistant"
["password", "sms_code", "hardware_key"] -> "mfa_phishing_resistant"
["biometric", "password"]             -> "mfa_phishable"
[]                                     -> "single_factor"
```

Constraints

- Deterministic, offline, standard library only.
- Follow the exact factor -> category mapping in `interface.md`.
- Unrecognized factor strings must not raise an error; they are simply ignored.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Optional

_CATEGORY = {
    "password": "knowledge",
    "pin": "knowledge",
    "security_question": "knowledge",
    "sms_code": "possession",
    "totp_code": "possession",
    "authenticator_app": "possession",
    "push_approved": "possession",
    "backup_code": "possession",
    "hardware_key": "possession",
    "fido2": "possession",
    "biometric": "inherence",
}

_PHISHING_RESISTANT = {"hardware_key", "fido2"}


def _category_of(factor: str) -> Optional[str]:
    return _CATEGORY.get(factor)


def solve(factors: List[str]) -> str:
    if not factors:
        return "single_factor"

    distinct_categories = set()
    has_phishing_resistant = False

    for factor in factors:
        category = _category_of(factor)
        if category is None:
            continue
        distinct_categories.add(category)
        if factor in _PHISHING_RESISTANT:
            has_phishing_resistant = True

    if len(distinct_categories) < 2:
        return "single_factor"

    if has_phishing_resistant:
        return "mfa_phishing_resistant"

    return "mfa_phishable"
```

4.2.c · Session security

3 pts · auto-graded on the site

Coding problem, 4.2.c: Classify session-security incidents

Skill: Mitigate Risk · **Maps to:** 4.2.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives short incident-report strings describing observed session behavior. Build the classifier that maps each description to the correct session-security category.

Categories:

- "hijacking", an already-authenticated session token is stolen (XSS read, network sniffing, malware) and reused by an attacker
- "fixation", an attacker supplies a session ID to the victim before login, then reuses that same ID after the victim authenticates
- "insecure_cookie", the session cookie is missing a secure attribute (HttpOnly, Secure, or SameSite) that would have prevented exposure or misuse
- "timeout_issue", a session remains valid far longer than it should because of a missing or misconfigured idle/absolute timeout
- "secure", the report describes a session that is properly protected (regeneration, secure flags, timeouts all present) with no vulnerability
- "unknown", nothing in the report matches any category

A report may match exactly one category. If no category matches, return "unknown".

Function contract

Implement `solve(report: str) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single category label string. Matching is case-insensitive substring detection using the indicator sets defined in `interface.md`.

Input

A single string describing observed session behavior on a device, browser, or server.

Output

One of: "hijacking", "fixation", "insecure_cookie", "timeout_issue", "secure", "unknown".

Examples

```
"Session token was sniffed over unencrypted HTTP and replayed by an attacker"        -> "hijacking"
"Attacker sent a link with a pre-set session id; victim logged in and id unchanged"  -> "fixation"
"Session cookie has no HttpOnly flag configured; automated scan flagged the issue"    -> "insecure_cookie"
"Logged-in kiosk left unattended for hours with no idle timeout configured"          -> "timeout_issue"
"Session id regenerated at login; cookie set Secure, HttpOnly, SameSite=Strict"      -> "secure"
"User changed their display name in account settings"                                -> "unknown"
```

Constraints

- Deterministic, offline, standard library only.
- Detection by case-insensitive substring match against indicator sets in `interface.md`.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(report: str) -> str:
    if not isinstance(report, str):
```

```
    return "unknown"

report_lower = report.lower()


fixation_indicators = [
    "fixation", "pre-set session", "attacker-supplied session",
    "session id unchanged after login", "id unchanged", "session id before",
    "known session id", "id was already set"
]
if any(ind in report_lower for ind in fixation_indicators):
    return "fixation"

hijacking_indicators = [
    "sniffed", "sniffing", "intercepted", "stolen session", "stole the session",
    "replayed", "captured token", "plain http", "plaintext",
    "read by an injected script", "read via injected", "malware read the",
    "xss", "reused by an attacker"
]
if any(ind in report_lower for ind in hijacking_indicators):
    return "hijacking"

insecure_cookie_indicators = [
    "missing httponly", "no httponly", "httponly flag", "missing secure flag",
    "no secure flag", "missing samesite", "no samesite", "cookie has no",
    "cookie is missing", "readable from client-side javascript",
    "not set to httponly"
]
if any(ind in report_lower for ind in insecure_cookie_indicators):
    return "insecure_cookie"

timeout_indicators = [
    "no idle timeout", "no timeout", "without a timeout", "left unattended",
    "no absolute timeout", "never expires", "does not expire",
    "session never times out", "unattended for", "abandoned", "kiosk"
]
if any(ind in report_lower for ind in timeout_indicators):
    return "timeout_issue"

secure_indicators = [
    "regenerated at login", "session id regenerated", "secure, httponly",
    "all secure flags", "properly secured", "secure flags set",
    "set secure", "set httponly", "samesite=strict",
    "session expired automatically"
]
if any(ind in report_lower for ind in secure_indicators):
    return "secure"

return "unknown"
```

4.2.d · Least privilege and access control models

3 pts · auto-graded on the site

Coding problem, 4.2.d: Access-request auditor

Skill: Mitigate Risk · **Maps to:** 4.2.B, 4.2.C · **Language:** Python 3 (pytest)

Problem

A security team reviews access requests before granting them. Each request either has a business justification attached or it does not; the requester's access may or may not have been reviewed recently; and the requested access may be scoped to a

temporary time window or be standing (permanent). Build the auditor function that applies the least-privilege triage rules to a single request and returns the decision.

Function contract

Implement `def solve(request: dict) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dict with keys `current_role` (str), `requested_access` (str), `justification_present` (bool), `time_bound` (bool or None), `last_reviewed_days_ago` (int). Optional keys default as documented in `interface.md` (missing `justification_present` defaults to `False`; missing `time_bound` defaults to `False`; missing `last_reviewed_days_ago` defaults to `0`).

Output

One of the strings: `"approve"`, `"deny_no_justification"`, `"flag_privilege_creep"`, `"approve_time_bound"`.

Decision rule (apply in this exact order, first match wins)

1. No justification (`justification_present` is `False`) → `"deny_no_justification"`. 2. Else, stale review (`last_reviewed_days_ago > 365`) → `"flag_privilege_creep"`. 3. Else, time-bound request (`time_bound` is `True`) → `"approve_time_bound"`. 4. Else → `"approve"`.

Rule 1 is checked before rule 2, a request with no justification is denied even if it is also years overdue for review or time-bound. The staleness threshold is 365 days, strictly greater than (365 itself is still considered current).

Examples

```
{"current_role": "Marketing Intern", "requested_access": "prod-db-write",
 "justification_present": False, "time_bound": False, "last_reviewed_days_ago": 10}
    -> "deny_no_justification"

{"current_role": "HR Analyst", "requested_access": "payroll-admin",
 "justification_present": True, "time_bound": False, "last_reviewed_days_ago": 800}
    -> "flag_privilege_creep"

{"current_role": "On-call Engineer", "requested_access": "prod-ssh",
 "justification_present": True, "time_bound": True, "last_reviewed_days_ago": 5}
    -> "approve_time_bound"

{"current_role": "Analyst", "requested_access": "reporting-read",
 "justification_present": True, "time_bound": False, "last_reviewed_days_ago": 30}
    -> "approve"
```

Constraints

- Deterministic, offline, standard library only.
- Missing optional keys must not raise, use the documented defaults.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(request: dict) -> str:
    justification_present = bool(request.get("justification_present", False))
    time_bound = request.get("time_bound", False)
    last_reviewed_days_ago = request.get("last_reviewed_days_ago", 0)

    if not justification_present:
        return "deny_no_justification"

    if last_reviewed_days_ago > 365:
        return "flag_privilege_creep"

    if time_bound is True:
        return "approve_time_bound"

    return "approve"
```